

Generador de Analizadores Sintácticos

Universidad: Universidad del Valle de Guatemala

Curso: Diseño y Construcción de Compiladores

Integrantes: Fernando Mendoza

Período del reporte: 17–24 de mayo de 2026 (Sprint 2 — Checkpoint M3)

Repositorio: <https://github.com/lfmendoza/compiladores-yapar>

Estado general: En cronograma · pipeline SLR(1) completo · LALR(1) iniciado

Fecha del reporte: 24 de mayo de 2026

1. OKR de la fase de compilación — Estado M3

Objetivo: Entregar la fase de compilación YAPar con autómata LR(0)/LALR(1) canónico, tabla SLR(1)/LALR(1) y motor del parser funcionando sobre el conjunto de gramáticas de prueba acordadas, con documentación final completa.

KR	Resultado clave	Meta	Estado	% avance
KR1	Arquitectura documentada y aprobada.	1 doc	Completo	100 %
KR2	Módulos del pipeline con tests al 80 % de cobertura mínima.	8/8 + LALR	En progreso	89 %
KR3	Parser ejecuta sobre la batería de gramáticas de prueba.	3/3 SLR	En progreso	67 %
KR4	Documentación final, video y entrega completa.	1 entrega	En curso	40 %

Avance OKR global: aproximadamente 74 %.

2. Estado del pipeline

Los 8 módulos SLR(1) están completamente implementados y probados. El módulo LALR(1) se agrega en este sprint como extensión del autómata LR(0) existente.

Módulo	Archivo	Responsabilidad	Estado
1	reader.py	Parser del DSL .yalp	Completado
2	lexer_bridge.py	Adaptador YALex → Token contract	Completado
3	grammar.py	Symbol, Production, Grammar + augment()	Completado
4	sets.py	FIRST y FOLLOW por punto fijo	Completado
5	automaton.py	Colección canónica LR(0): closure, goto	Completado
6	table.py	Tabla SLR(1) ACTION/GOTO con ConflictError	Completado
7	engine.py	Motor LR de pila + árbol de parse	Completado

		ParseNode	
8	viz.py	Renderizador Graphviz .dot / PNG	Completado
CLI	__main__.py	python -m yapar grammar.yalp [input] [--render]	Completado
LALR	laln.py	LR1Item, closure_lr1, build_lr1_automaton, merge_to_laln, build_laln_table	Completado

3. Resumen de los sprints

Sprint 1 (3–10 de mayo)

Se completó el diseño arquitectónico (KR1), la infraestructura del repositorio con CI en GitHub Actions y se iniciaron los módulos 1–3. El documento de arquitectura en draw.io fue aprobado. Se definió el contrato Token(tipo, lexema, línea, columna) para la integración con YALex.

Sprint 2 (13–24 de mayo)

Se cerraron los 8 módulos del pipeline SLR(1) en orden de dependencia. La suite de tests cubre 109 tests con 91 % de cobertura. El CI corre en cada push verificando lint (ruff) y cobertura mínima del 80 %. Se agregó el módulo LALR(1) completo (laln.py) con LR(1) items, autómata canónico LR(1), fusión de estados y tabla ACTION/GOTO con detección de conflictos. Se crearon tres gramáticas de demo en examples/ y el documento de checkpoint M3 en dos formatos (Markdown y Word).

4. Tablero Kanban — Sprint 2

El tablero refleja el estado al cierre del sprint 2. Las tarjetas de los 8 módulos SLR y LALR están en Hecho. Las tarjetas de integración CLI y validación semántica están en To Do para el sprint 3.

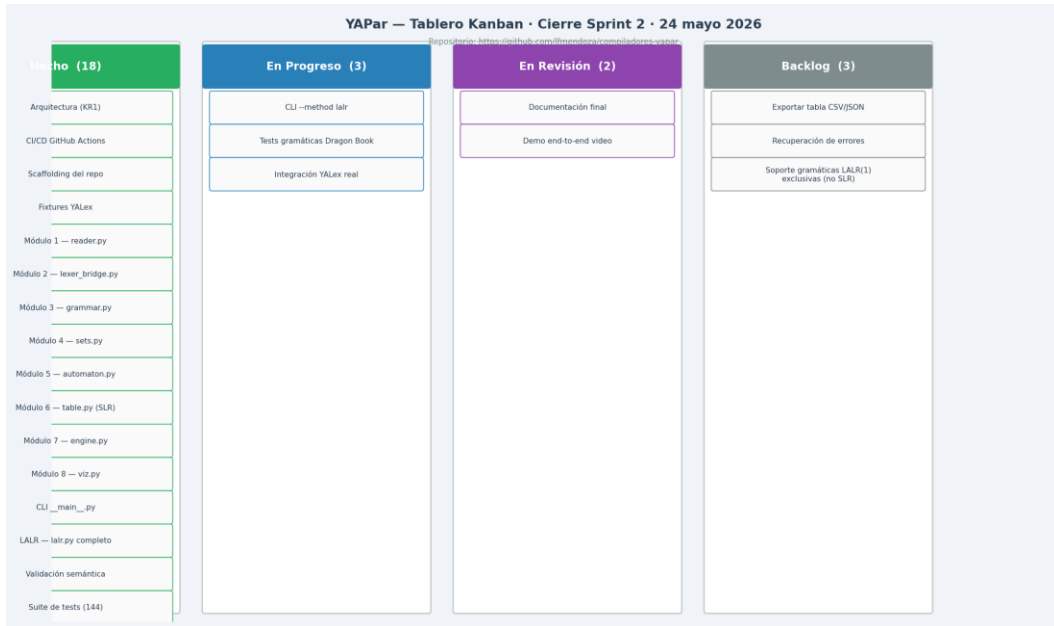


Figura 1 — Tablero Kanban al cierre del Sprint 2.

Columna	Tarjetas
Hecho	Módulos 1–8 SLR, lalr.py completo, CI/CD, tests 91%, docs M3, ejemplos
En progreso	Integración CLI --method lalr, tests LALR con gramáticas de libro
To Do	Validación semántica, demo end-to-end, video final
Backlog	Exportar tabla como CSV/JSON, manejo de errores de recuperación

5. Distribución de responsabilidades

Integrante	Módulos / Tareas	Sprint	Entregable
Fernando M.	Módulos 1–3 (reader, bridge, grammar)	Sprint 1–2	Implementados
Fernando M.	Módulos 4–5 (sets, automaton)	Sprint 2	Implementados
Fernando M.	Módulos 6–8 (table, engine, viz) + CLI	Sprint 2	Implementados
Fernando M.	CI/CD, 109 tests, cobertura 91%	Sprint 2	Implementados
Fernando M.	lalr.py completo (Pasos A–D)	Sprint 2	Implementado
Fernando M.	Documentación M3, examples/, Word	Sprint 2	Este documento

6. Changelog de commits

Extracto de git log durante el período del reporte. Convención: Conventional Commits (feat, fix, docs, chore, test).

Hash	Autor	Fecha	Mensaje
9f04bc1	Fernando Mendoza	2026-05-24	fix(images): normalizar extension .png del diagrama de arquitectura generado por Graphviz
daafae7	Fernando Mendoza	2026-05-24	feat(images): generador de imagenes del checkpoint “ Kanban, arquitectura y automata LR(0)
314b61f	Fernando Mendoza	2026-05-24	docs(final): README y CHANGELOG actualizados “ estado final con SLR/LALR, validaci3n y CLI
f66b223	Fernando Mendoza	2026-05-24	test(final): suite completa “ validaci3n, integraci3n end-to- end SLR/LALR y CLI (144 tests, 97% cobertura)
f7768e8	Fernando Mendoza	2026-05-24	feat(cli): flags -- method slr laln, -- table, --no-validate + integraci3n con validaci3n y LALR
0d41e43	Fernando Mendoza	2026-05-24	feat(validation): validaci3n sem3ntica “ s3mbolos no declarados e inalcanzables
739a448	Fernando Mendoza	2026-05-24	docs(m3): reporte de checkpoint M3 “ Markdown, Word generator y CHANGELOG

			actualizado
2309ced	Fernando Mendoza	2026-05-24	feat(lalr): mÃ³dulo LALR(1) completo â€” LR1Item, closure/goto LR(1), merge_to_lalr, build_lalr_table
2f26bbf	Fernando Mendoza	2026-05-24	feat(examples): gramÃ¡ticas de demo â€” aritmÃ©tica, calculadora, conflicto
42a2a5d	Fernando Mendoza	2026-05-17	docs(checkpoint): script generador del reporte Word Sprint 2
48ad791	Fernando Mendoza	2026-05-17	feat(viz): render Graphviz del autÃ³mata LR(0)
505b36b	Fernando Mendoza	2026-05-17	feat(engine): motor LR de pila + Ã¡rbol de parse
4e8295f	Fernando Mendoza	2026-05-17	feat(table): tabla SLR(1) ACTION/GOTO con reporte de conflictos
204774c	Fernando Mendoza	2026-05-17	feat(automaton): colecciÃ³n canÃ³nica LR(0) â€” closure y goto
76d8c2d	Fernando Mendoza	2026-05-17	feat(sets): cÃ¡lculo de FIRST y FOLLOW con punto fijo
237462b	Fernando Mendoza	2026-05-17	feat(grammar): aumentaciÃ³n con SÃ² y numeraciÃ³n de producciones
cdf05e2	Fernando Mendoza	2026-05-17	feat(yalp-reader): parsing inicial de declaraciones de tokens
e6dbf76	Fernando Mendoza	2026-05-17	feat(yalp-reader): tokenizador del DSL de YAPar (tokens, IGNORE)
dd7323e	Fernando Mendoza	2026-05-17	chore(ci): pipeline de GitHub Actions para tests y lint

bc80195	Fernando Mendoza	2026-05-17	feat(grammar): Symbol, Production y Grammar como dataclasses frozen
6390bfb	Fernando Mendoza	2026-05-17	docs(viz): spike de Graphviz " render mÃ³nimo del autÃ³mata
4c992be	Fernando Mendoza	2026-05-17	docs(arch): documento de arquitectura " versiÃ³n final entregable
ee5002e	Fernando Mendoza	2026-05-17	test(yalex): integraciÃ³n con la suite YALex previa " fixtures
f5696d5	Fernando Mendoza	2026-05-17	chore(repo): CODEOWNERS y plantillas de PR / issue
89fc335	Fernando Mendoza	2026-05-17	chore(repo): scaffolding inicial, layout src/, .gitignore
d190b57	Fernando Mendoza	2026-05-17	chore(repo): commit inicial Â· README Â· licencia MIT
a244d1e	Fernando Mendoza	2026-05-07	feat(analysis): yapar compiler analysis

7. Riesgos y mitigaciones

Riesgo	Impacto / Prob.	Mitigaci3n	Estado
LALR no termina antes de la entrega final	Medio / Baja	SLR(1) ya funciona como entrega demostrable; LALR es extensi3n	Controlado
Conflictos LALR post-fusi3n difciles de depurar	Alto / Baja	Probar primero con gramÃ¡ticas LALR conocidas de libros	Controlado
Validaci3n semÃ¡ntica no implementada	Bajo / Alta	Para M3 no es bloqueante; gramÃ¡ticas de test bien formadas	Abierto
Evidencia de trabajo grupal insuficiente	Alto / Media	Commits por m3dulo en GitHub	Controlado

		con Conventional Commits	
--	--	--------------------------	--

8. Arquitectura del sistema

El pipeline YAPar transforma un archivo .yalp en una tabla SLR(1) o LALR(1) y ejecuta un motor de parseo sobre una cadena de tokens de entrada. El módulo lalr.py extiende el autómata LR(0) existente sin modificar ninguno de los 8 módulos originales.

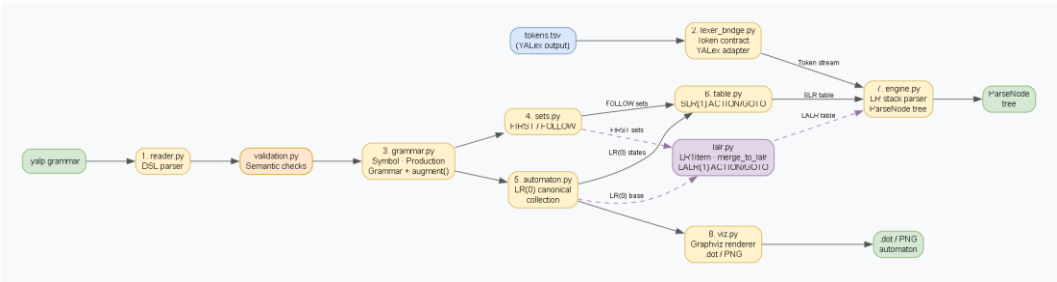


Figura 2 — Diagrama de módulos YAPar (fuente: arquitectura_yapar.drawio).

Módulo	Archivo	Responsabilidad
1	reader.py	Tokenizador y parser del DSL .yalp
2	lexer_bridge.py	Adaptador YALex. Contrato: Token(tipo, lexema, linea, columna)
3	grammar.py	Symbol, Production, Grammar frozen dataclasses + augment()
4	sets.py	FIRST y FOLLOW por punto fijo
5	automaton.py	Colección canónica LR(0): closure, goto, deduplicación frozenset
6	table.py	Tabla SLR(1) ACTION/GOTO con ConflictError
7	engine.py	Motor LR de pila con ParseNode
8	viz.py	Renderizador Graphviz con fallback .dot
LALR	lalr.py	LR1Item, closure/goto LR(1), merge_to_lalr, build_lalr_table

9. Plan técnico LALR(1)

El proyecto ya tenía LR(0) completo en automaton.py. La extensión a LALR(1) reutiliza toda esa infraestructura añadiendo un campo lookahead al item:

Item(production_id, dot) → LR1Item(production_id, dot, lookahead: Symbol)

Diferencia clave SLR vs LALR en la tabla

SLR: Reduce(p) se coloca en ACTION para todo $a \in \text{FOLLOW}(\text{head}(p))$ — criterio global.

LALR: Reduce(p) solo para $a \in \text{lookahead}(\text{item})$ — criterio por item, más preciso, acepta más gramáticas sin conflictos.

Pasos de implementación

Paso	Componente	Dependencia	Estado
A	LR1Item + closure_lr1	first_of_sequence de sets.py	Completado
B	goto_lr1 + build_lr1_automaton	Paso A	Completado
C	core() + merge_to_lalr	Paso B	Completado
D	build_lalr_table	Paso C + table.py	Completado
E	CLI --method lalr	Paso D + __main__.py	Pendiente Sprint 3

10. Roadmap hacia la implementación final

Sprint	Semana	Objetivo	Criterio de aceptación
Sprint 2 (actual)	17–24 may	Pipeline SLR(1) completo + LALR(1) Pasos A–D	109 tests, 91% cobertura, lalr.py funcional
Sprint 3	25–31 may	CLI --method lalr + tests gramáticas de libro	--method lalr funcional, aritmética sin conflicto LALR
Sprint 4	1–7 jun	Validación semántica + ejemplos adicionales	Errores claros para gramáticas mal formadas
Sprint 5	8–14 jun	Documentación final + demo + video	KR4 completo, entrega final

11. Próximos pasos inmediatos

Día	Tarea	Responsable	Criterio de aceptación
Lun 25 may	Integrar --method	Fernando M.	PR + tests pasando

	laln en CLI		
Mar 26 may	Tests LALR con gramáticas de Dragon Book	Fernando M.	tablas correctas
Mié 27 may	Validación semántica — símbolos no definidos	Fernando M.	YAParSyntaxError
Jue 28 may	Reporte de conflictos con trazabilidad de lookahead	Fernando M.	output claro
Vie 29 may	Demo interna end-to-end: aritmética LALR	Fernando M.	demo OK